1 Extending Non-Leaking Complex Numbers to C^2

The C^2 algebraic structure may offer greater cryptographic potential than initially anticipated. The concept of non-PFZ-leaking complex numbers can potentially be extended to create non-leaking C^2 structures. Specifically, the architecture can be designed such that whenever a multiplicative inverse does not exist, the norm of the C^2 number reliably evaluates to $(0, 0)$. This property would prevent the leakage of the z/z' encoding and, by extension, the Prime Factorization (PFZ) of the modulus.

A logical extension of this principle implies an encryption mapping structured as follows:

$$[r_1, r_2, m, m] \longrightarrow C^2 \tag{1}$$

In this scenario, a plaintext of zero (resulting in $[r_1, r_2, 0, 0]$) would inherently produce a norm of $(0, 0)$. For this to remain secure, it must be mathematically impossible for the two m components to assume differing values.

1.1 Derivation of Basis Vectors

Assuming $x = (p + 1)/2$ and $y = \sqrt{-x^2}$, the derivation of the C^2 basis vectors is formulated as follows:

$$\begin{aligned}
(x, 0, y, 0) &\longrightarrow (x, y) \longrightarrow (x^2, xy, xy, y^2) && (2)\\
&\longrightarrow (x, -y) \longrightarrow (x^2, -xy, xy, -y^2) && (3)\\
(x, 0, -y, 0) &\longrightarrow (x, y) \longrightarrow (x^2, xy, -xy, -y^2) && (4)\\
&\longrightarrow (x, -y) \longrightarrow (x^2, -xy, -xy, y^2) && (5)
\end{aligned}$$

These resulting tuples serve as the basis vectors for the space. Combining specific vectors (e.g., b_0 with b_1, or b_2 with b_3) successfully isolates the original structures $(x, 0, y, 0)$ or $(x, 0, -y, 0)$. This mechanism establishes the foundation for the encryption process.

Under this construction, plaintext zeros are ostensibly secure. If $m = 0$, the absence of an inverse element corresponds to an absolute norm $|c| = (0, 0)$, which protects both the PFZ and the underlying z/z' encoding.

2 Cryptanalysis: The Known Zero-Plaintext Attack

A critical security question remains: If an adversary possesses a known zero-plaintext ($m = 0$), is it truly impossible to compute the basis vectors or the hidden parameters x and y?

Given a known null-state, an attacker might attempt to solve the following System of Linear Equations (SLE):

$$r_1 \cdot b_2 + r_2 \cdot b_3 = c \tag{6}$$

Initially, this appears unsolvable because the random scalars r_1 and r_2 are unknown. However, a deeper structural analysis reveals a fatal vulnerability within the xy cross-terms.

2.1 Exploiting the Cross-Terms

The linear combination expands to:

$$r_1 \cdot (x^2, -xy, -xy, y^2) + r_2 \cdot (x^2, xy, -xy, -y^2) \tag{7}$$

Let the observed ciphertext be denoted as $c = (?, i, j, ?)$. Extracting the equations for the known components i and j yields:

$$r_1 xy + r_2 xy = i \tag{8}$$
$$r_1 xy - r_2 xy = j \tag{9}$$

Solving this sub-system allows the adversary to extract the isolated products:

- $r_1 \cdot xy$
- $r_2 \cdot xy$

2.2 The Squaring Attack

At first glance, recovering r_1, r_2, or xy individually from these products seems impossible. However, this defense is easily bypassed.

By homomorphically squaring the ciphertext c, the adversary generates a new set of equations. The squared cross-terms provide the adversary with:

- $r_1^2 \cdot xy$
- $r_2^2 \cdot xy$

By combining the linear terms $(r \cdot xy)$ with the squared terms $(r^2 \cdot xy)$, the adversary gains sufficient information to fully resolve r_1 and r_2, thereby breaking the encryption scheme entirely.